

Task 1: Test strategy

Test no.	Component to be tested	Type of test to be carried out	Prerequisites and dependencies
1	Sign up – ensure that a user account is added to the database when submitted and that a user is correctly redirected to login.	Normal – test for any type of unspotted syntax and logic errors that effects the codes ability to run.	Requires a functioning database with a corresponding customer table to insert data into (see 2 on design document, Er diagram) (see 9 on design document for test data)
2	Sign up – ensure that users can create an account that meets the boundaries of each input on either side.	Extreme – test the boundaries of different inputs, eg. what happens when you go to the character limit or enter the minimum amount of characters	Requires a functioning database with a corresponding customer table to insert data into (see 2 on design document, Er diagram)
3	Sign up – ensures users cannot create accounts with incorrect data (eg doesn't meet boundaries, blank fields, cannot register already existing accounts etc)	Erroneous – test how the system handles incorrect data eg trying to register accounts that already exist, submitting empty fields, dates that don't make sense etc.	Requires a functioning database with a corresponding customer table to insert data into (see 2 on design document, Er diagram)
4	Login – ensuring that users can login to pre- created	Normal – test for any type of unspotted syntax and	Requires a functioning database with a corresponding

	accounts that are in the systems database	logic errors that effects the codes ability to run.	customer table to select data from (see 2 on design document, Er diagram) and a function sign-up page to add user data too. (see 9 on design document for test data)
5	Login – ensuring that users can login to pre- created accounts that are in the systems database	Extreme – test the boundaries of different inputs, eg. what happens when you go to the character limit or enter the minimum amount of characters	Requires a functioning database with a corresponding customer table to select data from (see 2 on design document, Er diagram) and a function sign-up page to add user data too.
6	Login – ensuring that users can login to pre- created accounts that are in the systems database	Erroneous – test how the system handles incorrect data eg trying to register accounts that already exist, submitting empty fields, dates that don't make sense etc.	Requires a functioning database with a corresponding customer table to select data from (see 2 on design document, Er diagram) and a function sign-up page to add user data too.
7	Logout – ensuring users can successfully logout of the system	Ensuring that the session ends and they cannot access any pages that requires login cannot be accessed after logout.	Requires a functioning database (see 2 on design document, Er diagram), register and login system and a way to logout of the system

			accessible to the user.
8	Testing to ensure pages that require a user to be logged in to access is not accessible for users not logged in/ without accounts	Testing to ensure that users that are not logged into accounts cannot access or insert data to the database for security purposes (preventing attacks (see risks 13 on proposal document), ensuring activity around the database can be audited and linked to a user account) .	Requires a functioning database (see 2 on design document, Er diagram), register and login system. Pages that should redirect users that aren't logged in (eg if they try access the booking page, they will be asked to login to an account).
9	Audit system – ensuring that when a logged in user completes an action, it is recorded in an audit table in the database.	Testing to ensure the audit system works as intended to ensure that all activity (especially when interacting with the database) is audited for security purposes and to prevent users breaching the system (see 13 on proposal document, risks)	Requires a functioning database with an audit table that links to a user account in a separate table (see 2 on design document, Er diagram).
10	Adding items to digital basket – ensuring that when items are added to the basket, they are stored in session so users don't lose their baskets contents	Normal – test for any type of unspotted syntax and logic errors that effects the codes ability to run.	Requires a functioning database with a table of different products available to select from (see 2 on design document, Er diagram).

	when they refresh the page (eg to search for a different product)		A functioning digital basket that stores users product selection and its quantities. (see 9 on design document for test data)
11	Adding items to digital basket – ensuring that an extreme amount of items can be added to the basket without losing basket content. (based on available stock)	Extreme – test the extremes of different amounts of items in the basket. Ensuring that it doesn't allow the user to add more then what is available and allowing users to add the minimum requirement for each item	Requires a functioning database with a table of different products available to select from (see 2 on design document, Er diagram). A functioning digital basket that stores users product selection and its quantities. (see 9 on design document for test data)
12	Adding items to digital basket – ensuring that users cannot add items to basket that are out of stock or adding an amount that goes beyond what stock is available.	Erroneous - test how the system reacts to incorrect data (eg. trying to add an item to basket that is out of stock, trying to add more of an item then what is available etc)	Requires a functioning database with a table of different products available to select from (see 2 on design document, Er diagram). A functioning digital basket that stores users product selection and its quantities.

			(see 9 on design document for test data)
13	Stock levels – ensuring that stock available is shown to the user (especially in cases where less than 10 remain) and that users cannot purchase more stock than what is available	Testing how stock levels change after purchasing different amounts of different items. Ensuring they are up to date with stock available (eg staff adding stock after each delivery showing how much is available, set amount each day being available etc)	A functioning database that allows staff users to update the amount of stock available for each product (see 2 on design document, Er diagram). (see 9 on design document for test data)
14	Checkout: time slots – ensuring the system allows a user to select an available time slot/ date for their order to be delivered/ picked up successfully.	Normal – test for any type of unspotted syntax and logic errors that effects the codes ability to run.	A functioning database (see 2 on design document, Er diagram). A system that allows users to select a time slot for their order(separate ones for collection and delivery (see 6 on design document, Interface Designs))
15	Checkout: time slots - ensuring the system allows a user to select an available time slot/ date for their order to be delivered/ picked up for the extremes for different	Extreme – the extreme inputs for the time slots available. Ensuring that they are allowed to book them.	A functioning database (see 2 on design document, Er diagram). A system that allows users to select a time slot for their order(separate ones for collection and delivery (see 6

	slots (eg latest/ earliest slot)		on design document, Interface Designs))
16	Checkout: time slots – ensuring the system does not allow users to not select a time slot or pick a time slot that does not exist.	Erroneous – test how the system handles incorrect data (eg leaving the timeslot blank, trying to select a timeslot that isn't available/ doesn't exist)	A functioning database (see 2 on design document, Er diagram). A system that allows users to select a time slot for their order(separate ones for collection and delivery (see 6 on design document, Interface Designs))
17	Checkout: correct items, quantity and total – ensuring that when the user tries to check out the same items that were in the basket are displayed to the user as items they will be purchasing. With the correct quantities listed for each items with the correct total added as a final total and the total for each item.	Normal – test for any type of unspotted syntax and logic errors that effects the codes ability to run.	A functioning database (see 2 on design document, Er diagram). A system that correctly calculates a users total for each item they are purchasing in the correct quantity. As well as displaying the information on what they are buying and the quantity of each item before a user checks out.
18	Checkout: correct items, quantity and total – ensuring that the system responds correctly to the extremes of item amounts when it comes to	Extreme – test the extremes of different amounts of items in the basket. Ensuring that it calculates the users total correctly and displays the	A functioning database (see 2 on design document, Er diagram). A system that correctly calculates a users total for each item they are purchasing

	calculating totals and displaying the correct information to the user.	correct quantity of each item and the running total.	in the correct quantity. As well as displaying the information on what they are buying and the quantity of each item before a user checks out.
19	Checkout: correct items, quantity and total – ensuring the system doesn't not allow users to try to check out an empty order (effectively just steal stealing a time slot from a paying customer), or an order with the incorrect amount of stock then what is available.	Erroneous - test how the system reacts to incorrect data. Eg adding no items to basket and attempting to checkout, trying to add incorrect amounts of stock available etc)	A functioning database (see 2 on design document, Er diagram). A system that correctly calculates a users total for each item they are purchasing in the correct quantity. As well as displaying the information on what they are buying and the quantity of each item before a user checks out.
20	Alter Order – Allows users to add or remove items from an order before it is delivered/ collected (until around 24-48 hours before an order is collected/ delivered) successfully. Ensuring that the database is updated to reflect these changes.	Normal – test for any type of unspotted syntax and logic errors that effects the codes ability to run.	A functioning database that allows for an order in the database to be updated (see 2 on design document, Er diagram).
21	Alter Order – Allows users to add or remove items from an order before it is	Extreme – Test how the system reacts to adding or removing extreme	A functioning database that allows for an order in the database to be updated (see 2

	delivered/ collected (until around 24-48 hours before an order is collected/ delivered) to the extremes either end (buying adding all the stock or a large item with all the stock for that item) Ensuring that the database is updated to reflect these changes.	amounts of items and ensuring that it is updated to reflect that.	on design document, Er diagram).
22	Alter Order -Does not allow users to remove all the items in their order or add stock that isn't available to their order when altering it.	Erroneous – test how the system reacts to incorrect data. Eg (altering and emptying an order, adding unavailable stock etc)	A functioning database that allows for an order in the database to be updated (see 2 on design document, Er diagram).
23	Cancel Order – ensuring the system allows a user to delete their order and updating the system to reflect what stock quantity and time slot is available again.	Testing to see how an order reacts to a user cancelling their order (adding stock and timeslot back to being available etc)	A functioning database that allows for an order in the database to be deleted from the system (see 2 on design document, Er diagram).
24	Different Browsers – checking to make sure different browsers can successfully run the system and ensuring that certain browsers do not have	Testing compatibility with different browsers that users may use to interact with the system with.	A built system to test and a variety of different browsers to run the system on eg. popular browsers • google

	problems running the code correctly.		• firefox • opera • edge • bing • Less popular browsers • ecosia • vivaldi • different versions of other browsers eg waterfox
25	Page Load Time – testing to ensure the page loads quickly or in a reasonable time (NFR 2.2.2, see section 9 on proposal document)	Testing quality of code and runtime for different units of code. (unit testing)	A built system to unit test the different components of for their loading and run times.
26	Altering account details – allows a user to update their account details to reflect any changes (eg new email, address, adding and removing payment methods etc). This is important because data in systems needs to be kept up to date under GDPR/DPA (see section 11 of the proposal document)	Normal – test for any type of unspotted syntax and logic errors that effects the codes ability to run.	Requires a functioning database (see 2 on design document, Er diagram), register and login system and a way to alter users account details.

27	Altering account details – allows a user to update their account details to the boundaries of what is permitted by the system (eg character limits)	Extreme – testing to ensure the system can handle account details that reaches the boundaries of what the system allows (eg character limits)	Requires a functioning database (see 2 on design document, Er diagram), register and login system and a way to alter users account details.
28	Altering account details – Does not allow users to update their account information to incorrect data (eg removing essential data to login/ order products, changing emails to ones already taken in the system etc.)	Erroneous – ensuring the system does not allow users to update their account information with incorrect data (eg already in use emails, blank fields etc)	Requires a functioning database (see 2 on design document, Er diagram), register and login system and a way to alter users account details.
29	Unit integration – ensuring that different units of code function together as intended to create a fully fleshed out system.	Integration testing- testing to ensure that all the different sections of code function as intended at a reasonable speed (load time, see test 25)	A system with different units of code put together to test how they function as a whole.
30	Deleting account – Under GDPR/DPA (see section 11 of proposal document) users need to be able to successfully delete their accounts and remove their	Testing to ensure that accounts and the corresponding user data can be removed on request. Including from the systems database.	Requires a functioning database (see 2 on design document, Er diagram), register and login system and a way to delete user account details.

	data from the system on request.		
--	---	--	--